

Playwright Reporters

What are Playwright Reporters?

Playwright Reporters are used to display and format the results of your test execution.

After running your Playwright tests, reporters help you understand:

-  Which tests passed
-  Which tests failed
-  Which tests were skipped
-  Test execution time
-  Screenshots (if configured)
-  Videos (if configured)
-  Trace files (if configured)
-  Test execution summary

Think of reporters as different **ways of presenting the same test results**.

Types of Playwright Reporters

Playwright supports several built-in reporters:

 Reporter	Purpose
 HTML	Beautiful browser-based report
 List	Detailed console output (Default)
 Line	Compact single-line output
 Dot	Minimal terminal output
 JSON	Machine-readable JSON report
 JUnit	XML report for Jenkins & CI tools
 Custom	Build your own reporter



1. HTML Reporter



What is HTML Reporter?

The **HTML Reporter** generates an interactive and visually rich report that opens in your browser.

It is the **most commonly used Playwright reporter** because it is easy to understand and contains detailed information about every test.



HTML Report Includes

-  Passed Tests
 -  Failed Tests
 -  Skipped Tests
 -  Execution Time
 -  Screenshots
 -  Videos
 -  Trace Files
 -  Error Messages
 -  Summary Charts
-



Default Behavior

By default, Playwright:

- Creates an HTML report after execution.
 - Automatically opens the report **only if any test fails**.
-



Control When the Report Opens

You can control when Playwright automatically opens the HTML report.

Option	Behaviour
<code>open: 'never'</code>	 Never open automatically
<code>open: 'always'</code>	 Always open after execution
<code>open: 'on-failure'</code>	 Open only when a test fails (Default)

Configuration Example

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  reporter: [
    ['html', { open: 'never' }]
  ]
});
```

Save Report to a Custom Folder

Instead of using the default folder, you can store the report anywhere.

```
export default defineConfig({
  reporter: [
    ['html',
      {
        open: 'never',
        outputFolder: 'my-report'
      }
    ]
  ]
});
```

View HTML Report

Default Folder

```
npx playwright show-report
```

Custom Folder

```
npx playwright show-report my-report
```

Run Using CLI

```
npx playwright test --reporter=html
```

2. List Reporter (Default Reporter)

What is List Reporter?

The **List Reporter** displays **one line for every test** executed.

It is Playwright's **default reporter**.

Example Output

```
Running 5 tests

  ✓ Login Test
  ✓ Search Product
  ✗ Payment Test
  ✓ Logout Test
  ✓ Register Test
```

Advantages

-  Easy to read
 -  Shows each test individually
 -  Good for local development
-

Configuration

```
export default defineConfig({  
  reporter: 'list'  
});
```

CLI

```
npx playwright test --reporter=list
```



3. Line Reporter



What is Line Reporter?

The **Line Reporter** displays **only the latest running test** on a single line.

Instead of printing every test, it continuously updates the same line.

Example

```
Running Login Test...  
Running Search Test...  
Running Payment Test...
```

Only the last running test is visible.

Advantages

-  Cleaner output
 -  Less scrolling
 -  Useful for medium-sized test suites
-

Configuration

```
export default defineConfig({
  reporter: 'line'
});
```

CLI

```
npm playwright test --reporter=line
```

● 4. Dot Reporter

📖 What is Dot Reporter?

The **Dot Reporter** is the **smallest and most compact** reporter.

Instead of displaying test names, it prints **one symbol per test**.

Ideal for:

- CI/CD Pipelines
- Large automation suites
- Thousands of test cases

Example Output

All tests passed

```
.....
```

One test failed

```
....F....
```

Skipped test

..°.....

Retry

...x.....

Symbol Meaning

Symbol	Meaning
	Passed
	Failed
	Skipped
	Retried (or failed attempt during retries)

More Examples

All Passed

.....

One Failed

...F...

Skipped and Failed

..°...F

Why Use Dot Reporter?

Because it generates **very little console output**, making it perfect for:

-  Jenkins
 -  Azure DevOps
 -  GitHub Actions
 -  GitLab CI
 -  Large projects
-

Configuration

```
export default defineConfig({  
  reporter: 'dot'  
});
```

CLI

```
npx playwright test --reporter=dot
```

5. JSON Reporter

What is JSON Reporter?

The **JSON Reporter** stores test execution results in **JSON format**.

This format is useful when another application needs to read the results programmatically.

Common use cases:

-  Dashboards
 -  Analytics
 -  Automation scripts
 -  Integrations
-

Configuration

```
export default defineConfig({
  reporter: [
    ['json',
      {
        outputFile: 'results.json'
      }
    ]
  ]
});
```

CLI

```
npx playwright test --reporter=json
```



6. JUnit Reporter



What is JUnit Reporter?

The **JUnit Reporter** generates test results in **XML format**.

Many CI/CD tools understand JUnit XML.

Examples:

- Jenkins
 - Bamboo
 - TeamCity
 - Azure DevOps
-

Configuration

```
export default defineConfig({
  reporter: [
    ['junit',
      {
        outputFile: 'results.xml'
      }
    ]
  ]
});
```

CLI

```
npx playwright test --reporter=junit
```

7. Custom Reporter

What is a Custom Reporter?

Sometimes the built-in reporters are not enough.

Playwright allows you to create your **own reporter** by implementing the **Reporter interface**.

Example Custom Reporter

```
import type {
  FullConfig,
  FullResult,
  Reporter,
  Suite,
  TestCase,
  TestResult
} from '@playwright/test/reporter';

class MyReporter implements Reporter {

  onBegin(config: FullConfig, suite: Suite) {
    console.log(`Starting the run with ${suite.allTests().length} tests`);
  }

  onTestBegin(test: TestCase, result: TestResult) {
    console.log(`Starting test ${test.title}`);
  }

  onTestEnd(test: TestCase, result: TestResult) {
    console.log(`Finished test ${test.title}: ${result.status}`);
  }

  onEnd(result: FullResult) {
    console.log(`Finished the run: ${result.status}`);
  }

}

export default MyReporter;
```

Use the Custom Reporter

```
export default defineConfig({
  reporter: [
    ['./my-awesome-reporter.ts',
      {
        customOption: 'some value'
      }
    ]
  ]
});
```

CLI

```
npx playwright test --reporter="./myreporter/my-awesome-reporter.ts"
```



Configure Multiple Reporters Together

One of Playwright's best features is that **multiple reporters can run simultaneously**.

For example:

- Console Output
- HTML Report
- JSON Report
- XML Report
- Custom Report

—all generated in **one test execution**.

```
export default defineConfig({
  reporter: [

    ['list'],

    ['line'],

    ['dot'],

    ['html',
      {
        open: 'always',
        outputFolder: 'my-reports'
      }
    ],

    ['json',
      {
        outputFile: 'my-reports/results.json'
      }
    ],

    ['junit',
      {
        outputFile: 'my-reports/results.xml'
      }
    ],

    ['./tests/utils/CustomReporter.ts',
      {
        customOption: 'some value'
      }
    ]

  ]
});
```



Generate Allure Reports in Playwright



What is Allure Report?

Allure Report is one of the most popular reporting frameworks used with Playwright.

It provides a modern, interactive dashboard with rich details about test execution.



Allure Report Features

-  Dashboard
-  Execution statistics
-  Screenshots
-  Videos
-  Trace Files
-  Attachments
-  Error Details
-  Execution Timeline
-  Test Duration
-  Test History (when configured)
-  Trends across executions



Step 1: Install Allure CLI

Install the Allure Command Line Interface:

```
npm install -g allure-commandline --save-dev
```

Note: After installation, ensure the Allure executable is available in your system's **PATH** environment variable so you can run the `allure` command from the terminal.



Step 2: Install Allure Playwright Reporter

```
npm install -D allure-playwright
```

This installs the Playwright adapter as a development dependency.



Step 3: Configure Allure Reporter

Add the reporter to your `playwright.config.ts` file.

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  reporter: [
    ['allure-playwright', {
      outputFolder: 'allure-results'
    }]
  ]
});
```

Note: The reporter configuration should be provided as an array with the reporter name and its options.



Alternative: Use CLI

Without modifying the config:

```
npx playwright test --reporter=allure-playwright
```



Step 4: Generate HTML Report

After test execution, Allure stores raw results in:

```
allure-results
```

Generate the HTML report:

```
allure generate ./allure-results -o ./allure-report
```

Clean previous reports while generating:

```
allure generate ./allure-results -o ./allure-report --clean
```

Step 5: Open the Report

```
allure open ./allure-report
```

A browser window opens with a detailed Allure report.

Allure Report Contains

-  Passed Tests
 -  Failed Tests
 -  Skipped Tests
 -  Summary Dashboard
 -  Screenshots
 -  Videos
 -  Trace Files (if attached)
 -  Attachments
 -  Error Messages
 -  Graphs & Charts
 -  Timeline View
 -  Execution Duration
 -  Suites & Test Hierarchy
 -  History & Trends (when history is preserved)
-

Which Reporter Should You Use?

Reporter	Best For
 HTML	Everyday local development and detailed debugging
 List	Beginners and readable console output
 Line	Compact console output during development
 Dot	Large test suites and CI/CD pipelines
 JSON	Integrations, APIs, dashboards, and automation
 JUnit	Jenkins, Azure DevOps, Bamboo, TeamCity, and other CI servers
 Custom	Organization-specific reporting requirements
 Allure	Professional reporting with dashboards, attachments, history, and analytics

Key Takeaways

-  Reporters determine **how Playwright displays test results**.
-  **HTML Reporter** is the most popular choice for day-to-day debugging.
-  **List Reporter** is the default reporter and prints one line per test.
-  **Line Reporter** keeps console output compact by updating a single line.
-  **Dot Reporter** is ideal for CI/CD because it produces minimal output.
-  **JSON Reporter** generates machine-readable results for integrations.
-  **JUnit Reporter** creates XML reports for CI/CD tools like Jenkins.
-  **Custom Reporters** let you build reports tailored to your organization's needs.
-  **Allure Reports** provide rich dashboards with screenshots, videos, attachments, execution history, and detailed analytics, making them a preferred choice for enterprise automation projects.